

Chapter 42 — Digit Dynamic Programming

0. What You Will Learn in This Chapter

Digit Dynamic Programming, usually called **Digit DP**, is an advanced DP pattern used to count or optimize numbers under digit-based constraints.

Digit DP is used when a problem asks something like:

- Count numbers from 0 to n satisfying a property.
- Count numbers in range [low, high].
- Count numbers with digit sum in a range.
- Count numbers with no repeated digits.
- Count numbers with no consecutive equal digits.
- Count numbers divisible by k.
- Count numbers with some required digit pattern.
- Count binary numbers without consecutive ones.
- Count stepping numbers.
- Count beautiful numbers based on parity, digit sum, divisibility, or digit frequency.

Digit DP looks intimidating at first, but almost every problem follows the same structure:

```
dfs(position, tight, started, extra_state)
```

By the end of this chapter, you should understand:

- What Digit DP is
- When to use Digit DP
- Why normal DP does not directly work
- How to count numbers up to n
- How to convert range [low, high] into solve(high) - solve(low - 1)
- What tight means
- What leading zero handling means
- What started means
- How to memoize states
- How to track digit sum
- How to track used digits with a bitmask
- How to track modulo
- How to track previous digit
- How to solve hard range-counting problems

Because this is a hard DE Shaw-level chapter, we will use more explanations and more examples than a normal chapter.

Core problems covered in this chapter:

1. Count digit occurrences from 0 to n
2. Number of Digit One
3. Count Numbers With Unique Digits
4. Count Special Integers
5. Numbers With Repeated Digits
6. Numbers At Most N Given Digit Set
7. Count Integers With Digit Sum in Range
8. Non-negative Integers Without Consecutive Ones

9. Count Stepping Numbers in Range
 10. Number of Beautiful Integers in Range
 11. Rotated Digits
 12. Count numbers divisible by k
 13. Count numbers with exact digit sum
 14. Count numbers with no adjacent equal digits
 15. Digit DP template mastery
-

1. What Is Digit DP?

Digit DP is dynamic programming over the digits of a number.

Instead of iterating over every number from 1 to n, we build numbers digit by digit.

Example:

Suppose $n = 527$.

A valid number less than or equal to 527 can be built like this:

```
position 0: choose hundreds digit
position 1: choose tens digit
position 2: choose ones digit
```

At each position, the digit you can choose may depend on whether you are already below n or still equal to n so far.

For $n = 527$:

- At position 0, max digit is 5.
- If you choose 3, then the remaining digits can be anything from 0 to 9.
- If you choose 5, then position 1 is limited to at most 2.
- If you choose 52, then position 2 is limited to at most 7.

This upper-bound control is called **tight**.

2. Why Digit DP Exists

Many digit problems have huge ranges.

Example:

```
Count numbers from 1 to  $10^{18}$  whose digit sum is between 10 and 100.
```

You cannot iterate through all numbers.

But 10^{18} has only 19 digits.

Digit DP uses the number of digits, not the numeric range size.

So instead of:

```
O(n)
```

you often get:

```
O(number_of_digits * state_size * 10)
```

For 10^{18} , number_of_digits is only about 19.

That is why Digit DP is powerful.

3. Core Digit DP Idea

Most Digit DP problems define a function:

```
dfs(pos, tight, started, state)
```

Meaning:

```
pos      = current digit index
tight    = whether prefix is still equal to upper bound
started  = whether we have placed a non-leading-zero digit
state    = extra information needed by the problem
```

The answer is obtained by trying every digit allowed at the current position.

4. The Most Important Digit DP Terms

4.1 Position

pos tells which digit we are currently filling.

For number:

```
n = 527
digits = "527"
```

Positions:

```
pos 0 -> '5'
pos 1 -> '2'
pos 2 -> '7'
```

4.2 Tight

tight means:

Have all previous digits matched the upper bound exactly?

If tight is true, current digit cannot exceed the corresponding digit of n.

If tight is false, current digit can be 0 to 9.

Example:

n = 527

At position 0:

```
limit = 5
```

If choose digit 5:

```
nextTight = true
```

If choose digit 3:

```
nextTight = false
```

Because any number starting with 3 is already less than 527.

4.3 Started

started handles leading zeros.

When counting numbers up to 527, we often build all numbers with length 3:

```
001 -> 1
009 -> 9
045 -> 45
527 -> 527
```

Before we place the first non-zero digit, the number has not truly started.

This matters for constraints like:

- no repeated digits
- digit sum
- previous digit
- count of digits
- modulo

Usually, leading zeros should not count as real digits.

4.4 State

State is whatever extra information the problem needs.

Examples:

Problem	Extra State
Count digit sum	current sum
No repeated digits	used digit mask
Divisible by k	current remainder
No adjacent equal digits	previous digit
Count digit one	count of ones so far
Equal odd/even digits	odd count and even count
Stepping numbers	previous digit

5. Standard Digit DP Function Shape

The most common shape is:

```
long long dfs(int pos, bool tight, bool started, extra_state) {
    if (pos == digits.size()) {
        return valid_final_state ? 1 : 0;
    }

    int limit = tight ? digits[pos] - '0' : 9;

    long long answer = 0;

    for (int digit = 0; digit <= limit; digit++) {
        bool nextTight = tight && (digit == limit);
        bool nextStarted = started || digit != 0;
    }
}
```

```
        answer += dfs(pos + 1, nextTight, nextStarted,
updated_state);
    }

    return answer;
}
```

Important detail:

```
bool nextTight = tight && (digit == limit);
```

This works because `limit` equals the upper-bound digit when `tight == true`.

6. Range Counting Pattern

Most Digit DP functions count numbers from:

0 to n

Let:

`solve(n)` = count of valid numbers from 0 to n

Then count in range `[low, high]` is:

`solve(high) - solve(low - 1)`

This is extremely common.

Example:

```
answer = solve(high) - solve(low - 1);
```

For string ranges, you may need a helper to subtract one from a decimal string.

7. When to Use Digit DP

Use Digit DP when:

7.1 The Range Is Huge

Examples:

$1 \leq n \leq 10^{18}$

or:

low and high are strings of length 100

7.2 The Constraint Is Digit-Based

Examples:

- digit sum
- repeated digits
- digit parity
- digit product
- digit count
- adjacent digit rule
- modulo by k
- binary representation rules

7.3 You Need Count, Not List

Digit DP usually counts how many numbers satisfy a condition.

7.4 You Can Build the Number Left to Right

Most Digit DP builds from most significant digit to least significant digit.

8. When Not to Use Digit DP

Do not use Digit DP when:

8.1 n Is Small

If n is only 10^5 , brute force may be simpler.

8.2 The Problem Is Not Digit-Based

If the property has nothing to do with digits, Digit DP may be irrelevant.

8.3 Formula Is Easier

Some problems have direct math formulas.

Example:

Counting digit ones can be solved by a positional math formula, though Digit DP also works.

8.4 You Need Actual Numbers

If you need to generate all valid numbers, backtracking may be more natural.

9. Mental Models

9.1 Build a Number Digit by Digit

At every position, choose one digit.

9.2 Respect the Upper Bound

`tight` prevents building numbers greater than n .

9.3 Ignore Leading Zeros

`started` prevents leading zeros from affecting constraints.

9.4 Store Only Reusable States

Memoization usually works only when `tight == false`.

Why?

If `tight == true`, the remaining limit depends on the exact prefix of n .

If `tight == false`, the remaining positions are unrestricted, so the state is reusable.

10. Core Template 1 — Basic Digit DP Count

Count numbers from 0 to n .

```
class DigitDP {
public:
    string digits;
    vector<vector<long long>> memo;
```

```

long long dfs(int pos, bool tight) {
    if (pos == digits.size()) {
        return 1;
    }

    if (!tight && memo[pos][0] != -1) {
        return memo[pos][0];
    }

    int limit = tight ? digits[pos] - '0' : 9;
    long long answer = 0;

    for (int d = 0; d <= limit; d++) {
        bool nextTight = tight && (d == limit);
        answer += dfs(pos + 1, nextTight);
    }

    if (!tight) {
        memo[pos][0] = answer;
    }

    return answer;
}

long long solve(long long n) {
    digits = to_string(n);
    memo.assign(digits.size(), vector<long long>(1, -1));
    return dfs(0, true);
}
};

```

This just counts all numbers from 0 to n, so answer is $n + 1$. It is not useful by itself, but it teaches the structure.

11. Core Template 2 — Digit Sum DP

Count numbers from 0 to n with digit sum equal to target.

```

class Solution {
public:
    string digits;
    int target;
    vector<vector<long long>> memo;

    long long dfs(int pos, int sum, bool tight) {
        if (sum > target) {
            return 0;
        }

        if (pos == digits.size()) {

```

```

        return sum == target;
    }

    if (!tight && memo[pos][sum] != -1) {
        return memo[pos][sum];
    }

    int limit = tight ? digits[pos] - '0' : 9;
    long long answer = 0;

    for (int d = 0; d <= limit; d++) {
        bool nextTight = tight && (d == limit);
        answer += dfs(pos + 1, sum + d, nextTight);
    }

    if (!tight) {
        memo[pos][sum] = answer;
    }

    return answer;
}

long long countWithDigitSum(long long n, int requiredSum) {
    if (n < 0) {
        return 0;
    }

    digits = to_string(n);
    target = requiredSum;

    memo.assign(digits.size(), vector<long long>(target + 1, -
1));

    return dfs(0, 0, true);
}
};

```

12. Core Template 3 — Digit DP With Started

Count positive numbers from 1 to n with digit sum equal to target. Here leading zeros should not count as digits, but digit sum is unaffected by zeros. Still, we use started to exclude the empty number 0 if needed.

```

class Solution {
public:
    string digits;
    int target;

```

```

vector<vector<vector<long long>>> memo;

long long dfs(int pos, int sum, bool started, bool tight) {
    if (sum > target) {
        return 0;
    }

    if (pos == digits.size()) {
        return started && sum == target;
    }

    if (!tight && memo[pos][sum][started] != -1) {
        return memo[pos][sum][started];
    }

    int limit = tight ? digits[pos] - '0' : 9;
    long long answer = 0;

    for (int d = 0; d <= limit; d++) {
        bool nextStarted = started || d != 0;
        bool nextTight = tight && (d == limit);

        int nextSum = sum;

        if (nextStarted) {
            nextSum += d;
        }

        answer += dfs(pos + 1, nextSum, nextStarted, nextTight);
    }

    if (!tight) {
        memo[pos][sum][started] = answer;
    }

    return answer;
}

long long solve(long long n, int requiredSum) {
    if (n <= 0) {
        return 0;
    }

    digits = to_string(n);
    target = requiredSum;

    memo.assign(
        digits.size(),
        vector<vector<long long>>(target + 1, vector<long

```

```

long>(2, -1))
    );

    return dfs(0, 0, false, true);
}
};

```

13. Core Template 4 — Digit DP With Used Digit Mask

Used for numbers with no repeated digits.

```
long long dfs(int pos, int mask, bool started, bool tight)
```

Meaning:

```

pos = current digit position
mask = digits already used
started = have we started real number
tight = are we bounded by n

```

If digit d is already used:

```
if (mask & (1 << d)) continue;
```

Add digit:

```
nextMask = mask | (1 << d);
```

14. Major Digit DP Variations

Variation	State
Digit sum	pos, sum, tight
No repeated digits	pos, mask, started, tight
Divisible by k	pos, remainder, started, tight
Count digit x	pos, count, tight
No consecutive ones	pos, previousBit, tight
Stepping number	pos, previousDigit, started, tight
Beautiful integer	pos, oddCount, evenCount, remainder, started, tight
Range string counting	pos, sum, tight with string bounds

15. Full Problem Walkthrough 1 — Count Numbers With Exact Digit Sum

15.1 Problem Summary

Given n and $targetSum$, count numbers from 1 to n whose digit sum is exactly $targetSum$.

Example:

```
n = 25
targetSum = 7
```

Valid numbers:

```
7, 16, 25
```

Answer:

```
3
```

15.2 State

```
dfs(pos, sum, started, tight)
```

15.3 C++ Code

```
class Solution {
public:
    string digits;
    int target;
    vector<vector<vector<long long>>> memo;

    long long dfs(int pos, int sum, bool started, bool tight) {
        if (sum > target) {
            return 0;
        }

        if (pos == digits.size()) {
            return started && sum == target;
        }

        if (!tight && memo[pos][sum][started] != -1) {
            return memo[pos][sum][started];
        }

        int limit = tight ? digits[pos] - '0' : 9;
        long long answer = 0;

        for (int d = 0; d <= limit; d++) {
            bool nextStarted = started || d != 0;
            bool nextTight = tight && (d == limit);

            int nextSum = sum;
```

```

        if (nextStarted) {
            nextSum += d;
        }

        answer += dfs(pos + 1, nextSum, nextStarted, nextTight);
    }

    if (!tight) {
        memo[pos][sum][started] = answer;
    }

    return answer;
}

long long countExactDigitSum(long long n, int targetSum) {
    if (n <= 0) {
        return 0;
    }

    digits = to_string(n);
    target = targetSum;

    memo.assign(
        digits.size(),
        vector<vector<long long>>(target + 1, vector<long
long>(2, -1))
    );

    return dfs(0, 0, false, true);
}
};

```

15.4 Complexity

Let d be number of digits.

Time:

$O(d * \text{targetSum} * 2 * 10)$

Space:

$O(d * \text{targetSum} * 2)$

16. Full Problem Walkthrough 2 — Count Numbers in Range With Digit Sum

16.1 Problem Summary

Count numbers in [low, high] whose digit sum is exactly targetSum.

16.2 Key Pattern

```
answer = solve(high) - solve(low - 1)
```

16.3 C++ Code

```
class Solution {
public:
    string digits;
    int target;
    vector<vector<vector<long long>>> memo;

    long long dfs(int pos, int sum, bool started, bool tight) {
        if (sum > target) {
            return 0;
        }

        if (pos == digits.size()) {
            return started && sum == target;
        }

        if (!tight && memo[pos][sum][started] != -1) {
            return memo[pos][sum][started];
        }

        int limit = tight ? digits[pos] - '0' : 9;
        long long answer = 0;

        for (int d = 0; d <= limit; d++) {
            bool nextStarted = started || d != 0;
            bool nextTight = tight && (d == limit);

            int nextSum = sum;
            if (nextStarted) {
                nextSum += d;
            }

            answer += dfs(pos + 1, nextSum, nextStarted, nextTight);
        }

        if (!tight) {
            memo[pos][sum][started] = answer;
        }

        return answer;
    }

    long long solve(long long n) {
```

```

        if (n <= 0) {
            return 0;
        }

        digits = to_string(n);

        memo.assign(
            digits.size(),
            vector<vector<long long>>(target + 1, vector<long
long>(2, -1))
        );

        return dfs(0, 0, false, true);
    }

    long long countInRange(long long low, long long high, int
targetSum) {
        target = targetSum;
        return solve(high) - solve(low - 1);
    }
};

```

17. Full Problem Walkthrough 3 — LeetCode 233: Number of Digit One

17.1 Problem Summary

Given n , count how many times digit 1 appears in all numbers from 0 to n .

Example:

$n = 13$

Numbers:

1, 10, 11, 12, 13

Digit 1 appears:

6 times

17.2 Digit DP State

`dfs(pos, countOnes, tight)`

At the end, return `countOnes`.

Instead of counting valid numbers, each path contributes the number of ones in that number.

17.3 C++ Code

```

class Solution {
public:
    string digits;

```

```

vector<vector<long long>> memo;
vector<vector<bool>> seen;

long long dfs(int pos, int countOnes, bool tight) {
    if (pos == digits.size()) {
        return countOnes;
    }

    if (!tight && seen[pos][countOnes]) {
        return memo[pos][countOnes];
    }

    int limit = tight ? digits[pos] - '0' : 9;
    long long answer = 0;

    for (int d = 0; d <= limit; d++) {
        bool nextTight = tight && (d == limit);
        int nextCount = countOnes + (d == 1);

        answer += dfs(pos + 1, nextCount, nextTight);
    }

    if (!tight) {
        seen[pos][countOnes] = true;
        memo[pos][countOnes] = answer;
    }

    return answer;
}

int countDigitOne(int n) {
    if (n <= 0) {
        return 0;
    }

    digits = to_string(n);

    int maxDigits = digits.size();

    memo.assign(maxDigits, vector<long long>(maxDigits + 1, 0));
    seen.assign(maxDigits, vector<bool>(maxDigits + 1, false));

    return (int)dfs(0, 0, true);
}
};

```

17.4 Important Lesson

Not every Digit DP returns 1 at the end.

Sometimes the base case returns an accumulated value.

18. Full Problem Walkthrough 4 —

LeetCode 2376: Count Special Integers

18.1 Problem Summary

A special integer has no repeated digits.

Given n , count special integers from 1 to n .

Example:

$n = 20$

Special integers:

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 12, 13, 14, 15, 16, 17, 18, 19, 20

11 is not special.

18.2 State

`dfs(pos, mask, started, tight)`

mask tracks used digits.

18.3 C++ Code

```
class Solution {
public:
    string digits;
    long long memo[20][1 << 10][2];
    bool seen[20][1 << 10][2];

    long long dfs(int pos, int mask, bool started, bool tight) {
        if (pos == digits.size()) {
            return started ? 1 : 0;
        }

        if (!tight && seen[pos][mask][started]) {
            return memo[pos][mask][started];
        }

        int limit = tight ? digits[pos] - '0' : 9;
        long long answer = 0;

        for (int d = 0; d <= limit; d++) {
            bool nextTight = tight && (d == limit);

            if (!started && d == 0) {
                answer += dfs(pos + 1, mask, false, nextTight);
            } else {
                if (mask & (1 << d)) {
```

```

        continue;
    }

    answer += dfs(pos + 1, mask | (1 << d), true,
nextTight);
    }
}

if (!tight) {
    seen[pos][mask][started] = true;
    memo[pos][mask][started] = answer;
}

return answer;
}

int countSpecialNumbers(int n) {
    digits = to_string(n);
    memset(seen, false, sizeof(seen));

    return (int)dfs(0, 0, false, true);
}
};

```

18.4 Complexity

There are at most:

$\text{digits} * 1024 * 2$

states.

Each tries 10 digits.

Very efficient.

19. Full Problem Walkthrough 5 — LeetCode 1012: Numbers With Repeated Digits

19.1 Problem Summary

Given n , return how many numbers from 1 to n have at least one repeated digit.

19.2 Key Transformation

Instead of directly counting repeated-digit numbers, count special numbers first.

$\text{repeated} = n - \text{special}$

Because every number from 1 to n is either:

- no repeated digits
- has repeated digits

19.3 C++ Code

```

class Solution {
public:
    string digits;
    long long memo[20][1 << 10][2];
    bool seen[20][1 << 10][2];

    long long dfs(int pos, int mask, bool started, bool tight) {
        if (pos == digits.size()) {
            return started ? 1 : 0;
        }

        if (!tight && seen[pos][mask][started]) {
            return memo[pos][mask][started];
        }

        int limit = tight ? digits[pos] - '0' : 9;
        long long answer = 0;

        for (int d = 0; d <= limit; d++) {
            bool nextTight = tight && (d == limit);

            if (!started && d == 0) {
                answer += dfs(pos + 1, mask, false, nextTight);
            } else {
                if (mask & (1 << d)) {
                    continue;
                }

                answer += dfs(pos + 1, mask | (1 << d), true,
nextTight);
            }
        }

        if (!tight) {
            seen[pos][mask][started] = true;
            memo[pos][mask][started] = answer;
        }

        return answer;
    }

    int numDupDigitsAtMostN(int n) {
        digits = to_string(n);
        memset(seen, false, sizeof(seen));

        int special = dfs(0, 0, false, true);

        return n - special;
    }
};

```

```
}  
};
```

19.4 Key Lesson

Sometimes the easier Digit DP counts the complement.

20. Full Problem Walkthrough 6 — LeetCode 902: Numbers At Most N Given Digit Set

20.1 Problem Summary

You are given allowed digits as strings.

Count positive integers less than or equal to n that can be formed using only those digits.

Example:

```
digits = ["1","3","5","7"]  
n = 100
```

Valid:

```
1,3,5,7,11,13,15,17,31,33,...
```

20.2 State

```
dfs(pos, started, tight)
```

But when started is false, we can either:

- skip this position
- start with one allowed digit

20.3 C++ Code

```
class Solution {  
public:  
    string bound;  
    vector<int> allowed;  
    long long memo[20][2];  
    bool seen[20][2];  
  
    long long dfs(int pos, bool started, bool tight) {  
        if (pos == bound.size()) {  
            return started ? 1 : 0;  
        }  
  
        if (!tight && seen[pos][started]) {  
            return memo[pos][started];  
        }  
  
        int limit = tight ? bound[pos] - '0' : 9;  
        long long answer = 0;
```

```

        if (!started) {
            bool nextTight = tight && (0 == limit);
            answer += dfs(pos + 1, false, nextTight);
        }

        for (int d : allowed) {
            if (d > limit) {
                continue;
            }

            bool nextTight = tight && (d == limit);
            answer += dfs(pos + 1, true, nextTight);
        }

        if (!tight) {
            seen[pos][started] = true;
            memo[pos][started] = answer;
        }

        return answer;
    }

    int atMostNGivenDigitSet(vector<string>& digits, int n) {
        bound = to_string(n);
        allowed.clear();

        for (string d : digits) {
            allowed.push_back(d[0] - '0');
        }

        memset(seen, false, sizeof(seen));

        return (int)dfs(0, false, true);
    }
};

```

21. Full Problem Walkthrough 7 — LeetCode 2719: Count of Integers

21.1 Problem Summary

Given strings num1 and num2, count integers x such that:

num1 <= x <= num2

and digit sum is between:

min_sum and max_sum

Return answer modulo 1,000,000,007.

21.2 Why This Is Harder

Here bounds are strings.

So we need:

```
solve(num2) - solve(num1 - 1)
```

Need helper to subtract 1 from a string.

21.3 C++ Code

```
class Solution {
public:
    static const int MOD = 1000000007;

    string digits;
    int minSum;
    int maxSum;

    vector<vector<int>> memo;
    vector<vector<bool>> seen;

    int dfs(int pos, int sum, bool tight) {
        if (sum > maxSum) {
            return 0;
        }

        if (pos == digits.size()) {
            return sum >= minSum && sum <= maxSum;
        }

        if (!tight && seen[pos][sum]) {
            return memo[pos][sum];
        }

        int limit = tight ? digits[pos] - '0' : 9;
        long long answer = 0;

        for (int d = 0; d <= limit; d++) {
            bool nextTight = tight && (d == limit);
            answer += dfs(pos + 1, sum + d, nextTight);
            answer %= MOD;
        }

        if (!tight) {
            seen[pos][sum] = true;
            memo[pos][sum] = (int)answer;
        }

        return (int)answer;
    }
}
```

```

string subtractOne(string s) {
    int i = s.size() - 1;

    while (i >= 0 && s[i] == '0') {
        s[i] = '9';
        i--;
    }

    if (i >= 0) {
        s[i]--;
    }

    int start = 0;
    while (start + 1 < s.size() && s[start] == '0') {
        start++;
    }

    return s.substr(start);
}

int solve(string s) {
    digits = s;

    int maxPossibleSum = 9 * digits.size();

    memo.assign(digits.size(), vector<int>(maxSum + 1, 0));
    seen.assign(digits.size(), vector<bool>(maxSum + 1, false));

    return dfs(0, 0, true);
}

int count(string num1, string num2, int min_sum, int max_sum) {
    minSum = min_sum;
    maxSum = max_sum;

    int high = solve(num2);
    int low = solve(subtractOne(num1));

    int answer = high - low;

    if (answer < 0) {
        answer += MOD;
    }

    return answer;
}
};

```

21.4 Important Notes

This problem does not need started because digit sum with leading zeros behaves correctly.

Example:

007 has same digit sum as 7

So leading zeros are harmless.

22. Full Problem Walkthrough 8 — LeetCode 600: Non-negative Integers Without Consecutive Ones

22.1 Problem Summary

Given n , count non-negative integers less than or equal to n whose binary representation does not contain consecutive ones.

Example:

$n = 5$

Binary:

```
0 -> 0
1 -> 1
2 -> 10
3 -> 11 invalid
4 -> 100
5 -> 101
```

Answer:

5

22.2 Digit DP in Base 2

This is Digit DP, but digits are binary:

0 or 1

State:

`dfs(pos, prevBit, tight)`

22.3 C++ Code

```
class Solution {
public:
    string bits;
    int memo[40][2];
    bool seen[40][2];

    int dfs(int pos, int prevBit, bool tight) {
        if (pos == bits.size()) {
            return 1;
        }
        if (memo[pos][prevBit] != -1) {
            return memo[pos][prevBit];
        }
        int res = 0;
        int upper = tight ? bits[pos] - '0' : 1;
        for (int bit = 0; bit <= upper; bit++) {
            if (prevBit == 1 && bit == 1) continue;
            res += dfs(pos + 1, bit, tight && (bit == upper));
        }
        memo[pos][prevBit] = res;
        return res;
    }
};
```

```

    }

    if (!tight && seen[pos][prevBit]) {
        return memo[pos][prevBit];
    }

    int limit = tight ? bits[pos] - '0' : 1;
    int answer = 0;

    for (int bit = 0; bit <= limit; bit++) {
        if (prevBit == 1 && bit == 1) {
            continue;
        }

        bool nextTight = tight && (bit == limit);
        answer += dfs(pos + 1, bit, nextTight);
    }

    if (!tight) {
        seen[pos][prevBit] = true;
        memo[pos][prevBit] = answer;
    }

    return answer;
}

int findIntegers(int n) {
    bits.clear();

    for (int i = 31; i >= 0; i--) {
        if ((n >> i) & 1 || !bits.empty()) {
            bits.push_back(((n >> i) & 1) + '0');
        }
    }

    if (bits.empty()) {
        bits = "0";
    }

    memset(seen, false, sizeof(seen));

    return dfs(0, 0, true);
}
};

```

22.4 Key Lesson

Digit DP can work in any base, not just decimal.

23. Full Problem Walkthrough 9 — Count Numbers Divisible by k

23.1 Problem Summary

Given n and k, count numbers from 1 to n divisible by k.

We will solve it using Digit DP for learning, although simple division can solve this specific version.

23.2 State

```
dfs(pos, remainder, started, tight)
```

When we append digit d:

```
newRemainder = (remainder * 10 + d) % k
```

23.3 C++ Code

```
class Solution {
public:
    string digits;
    int k;
    vector<vector<vector<long long>>> memo;
    vector<vector<vector<bool>>> seen;

    long long dfs(int pos, int rem, bool started, bool tight) {
        if (pos == digits.size()) {
            return started && rem == 0;
        }

        if (!tight && seen[pos][rem][started]) {
            return memo[pos][rem][started];
        }

        int limit = tight ? digits[pos] - '0' : 9;
        long long answer = 0;

        for (int d = 0; d <= limit; d++) {
            bool nextStarted = started || d != 0;
            bool nextTight = tight && (d == limit);

            int nextRem = rem;

            if (nextStarted) {
                nextRem = (rem * 10 + d) % k;
            }

            answer += dfs(pos + 1, nextRem, nextStarted, nextTight);
        }
    }
};
```

```

        if (!tight) {
            seen[pos][rem][started] = true;
            memo[pos][rem][started] = answer;
        }

        return answer;
    }

    long long countDivisible(long long n, int divisor) {
        if (n <= 0) {
            return 0;
        }

        digits = to_string(n);
        k = divisor;

        memo.assign(
            digits.size(),
            vector<vector<long long>>(k, vector<long long>(2, 0))
        );

        seen.assign(
            digits.size(),
            vector<vector<bool>>(k, vector<bool>(2, false))
        );

        return dfs(0, 0, false, true);
    }
};

```

24. Full Problem Walkthrough 10 — LeetCode 2801: Count Stepping Numbers in Range

24.1 Problem Summary

A stepping number is a number where adjacent digits differ by exactly 1.

Examples:

```

12
21
123
321
8987

```

Invalid:

13
100

Given low and high as strings, count stepping numbers in range.

24.2 State

dfs(pos, prevDigit, started, tight)

If number has not started, we can place 0 and continue.

If started, next digit must satisfy:

abs(d - prevDigit) == 1

24.3 C++ Code

```
class Solution {
public:
    static const int MOD = 1000000007;

    string digits;
    int memo[105][11][2];
    bool seen[105][11][2];

    int dfs(int pos, int prev, bool started, bool tight) {
        if (pos == digits.size()) {
            return started ? 1 : 0;
        }

        if (!tight && seen[pos][prev][started]) {
            return memo[pos][prev][started];
        }

        int limit = tight ? digits[pos] - '0' : 9;
        long long answer = 0;

        for (int d = 0; d <= limit; d++) {
            bool nextTight = tight && (d == limit);

            if (!started && d == 0) {
                answer += dfs(pos + 1, 10, false, nextTight);
            } else if (!started) {
                answer += dfs(pos + 1, d, true, nextTight);
            } else {
                if (abs(d - prev) == 1) {
                    answer += dfs(pos + 1, d, true, nextTight);
                }
            }
        }

        answer %= MOD;

        if (!tight) {

```

```

        seen[pos][prev][started] = true;
        memo[pos][prev][started] = (int)answer;
    }

    return (int)answer;
}

string subtractOne(string s) {
    int i = s.size() - 1;

    while (i >= 0 && s[i] == '0') {
        s[i] = '9';
        i--;
    }

    if (i >= 0) {
        s[i]--;
    }

    int start = 0;
    while (start + 1 < s.size() && s[start] == '0') {
        start++;
    }

    return s.substr(start);
}

int solve(string s) {
    digits = s;
    memset(seen, false, sizeof(seen));

    return dfs(0, 10, false, true);
}

int countSteppingNumbers(string low, string high) {
    int highCount = solve(high);
    int lowCount = solve(subtractOne(low));

    int answer = highCount - lowCount;

    if (answer < 0) {
        answer += MOD;
    }

    return answer;
}
};

```

24.4 Why prev = 10?

Digits are 0 to 9.

So 10 is used as a sentinel meaning:

no previous digit yet

25. Full Problem Walkthrough 11 — LeetCode 2827: Number of Beautiful Integers in Range

25.1 Problem Summary

A number is beautiful if:

- it has equal number of odd and even digits
- it is divisible by k

Given low, high, and k, count beautiful integers in [low, high].

25.2 State

We need track:

```
pos
odd count
even count
remainder modulo k
started
tight
```

State:

```
dfs(pos, odd, even, rem, started, tight)
```

At the end:

```
started == true
odd == even
rem == 0
```

25.3 C++ Code

```
class Solution {
public:
    string digits;
    int K;

    map<tuple<int, int, int, int, bool>, int> memo;

    int dfs(int pos, int odd, int even, int rem, bool started, bool
tight) {
        if (pos == digits.size()) {
            return started && odd == even && rem == 0;
        }
    }
```

```

        if (!tight) {
            auto key = make_tuple(pos, odd, even, rem, started);
            if (memo.count(key)) {
                return memo[key];
            }
        }

        int limit = tight ? digits[pos] - '0' : 9;
        int answer = 0;

        for (int d = 0; d <= limit; d++) {
            bool nextTight = tight && (d == limit);

            if (!started && d == 0) {
                answer += dfs(pos + 1, odd, even, rem, false,
nextTight);
            } else {
                bool nextStarted = true;

                int nextOdd = odd + (d % 2 == 1);
                int nextEven = even + (d % 2 == 0);
                int nextRem = (rem * 10 + d) % K;

                answer += dfs(pos + 1, nextOdd, nextEven, nextRem,
nextStarted, nextTight);
            }
        }

        if (!tight) {
            auto key = make_tuple(pos, odd, even, rem, started);
            memo[key] = answer;
        }

        return answer;
    }

    int solve(int n) {
        if (n <= 0) {
            return 0;
        }

        digits = to_string(n);
        memo.clear();

        return dfs(0, 0, 0, 0, false, true);
    }

    int numberOfBeautifulIntegers(int low, int high, int k) {
        K = k;

```

```
        return solve(high) - solve(low - 1);
    }
};
```

25.4 Why Use map Here?

This has several state dimensions.

A fixed-size array is possible, but map makes the code easier to read.

In interviews, clarity is often more important first. Optimize after correctness.

26. Full Problem Walkthrough 12 — No Adjacent Equal Digits

26.1 Problem Summary

Count numbers from 1 to n where no two adjacent digits are equal.

Examples:

Valid:

```
12
101
1234
909
```

Invalid:

```
11
100
1223
```

26.2 State

```
dfs(pos, prevDigit, started, tight)
```

26.3 C++ Code

```
class Solution {
public:
    string digits;
    long long memo[20][11][2];
    bool seen[20][11][2];

    long long dfs(int pos, int prev, bool started, bool tight) {
        if (pos == digits.size()) {
            return started ? 1 : 0;
        }

        if (!tight && seen[pos][prev][started]) {
            return memo[pos][prev][started];
        }

        int limit = tight ? digits[pos] - '0' : 9;

        long long ans = 0;
        for (int d = 0; d <= limit; d++) {
            if (d == prev) continue;
            ans += dfs(pos + 1, d, started || d > 0, tight && d == limit);
        }

        if (!tight && seen[pos][prev][started]) {
            memo[pos][prev][started] = ans;
        }

        return ans;
    }
};
```

```

long long answer = 0;

for (int d = 0; d <= limit; d++) {
    bool nextTight = tight && (d == limit);

    if (!started && d == 0) {
        answer += dfs(pos + 1, 10, false, nextTight);
    } else {
        if (started && d == prev) {
            continue;
        }

        answer += dfs(pos + 1, d, true, nextTight);
    }

    if (!tight) {
        seen[pos][prev][started] = true;
        memo[pos][prev][started] = answer;
    }

    return answer;
}

long long countNoAdjacentEqual(long long n) {
    if (n <= 0) {
        return 0;
    }

    digits = to_string(n);
    memset(seen, false, sizeof(seen));

    return dfs(0, 10, false, true);
}
};

```

27. Full Problem Walkthrough 13 — LeetCode 788: Rotated Digits

27.1 Problem Summary

A number is good if after rotating each digit 180 degrees, it becomes a different valid number.

Valid rotating digits:

```

0 -> 0
1 -> 1
2 -> 5

```

5 -> 2
6 -> 9
8 -> 8
9 -> 6

Invalid digits:

3, 4, 7

A number is good if it contains at least one changing digit:

2, 5, 6, 9

27.2 State

dfs(pos, changed, started, tight)

At the end, valid if:

started == true
changed == true

27.3 C++ Code

```
class Solution {
public:
    string digits;
    int memo[20][2][2];
    bool seen[20][2][2];

    bool isValidDigit(int d) {
        return d == 0 || d == 1 || d == 2 || d == 5 || d == 6 || d
== 8 || d == 9;
    }

    bool isChangingDigit(int d) {
        return d == 2 || d == 5 || d == 6 || d == 9;
    }

    int dfs(int pos, bool changed, bool started, bool tight) {
        if (pos == digits.size()) {
            return started && changed;
        }

        if (!tight && seen[pos][changed][started]) {
            return memo[pos][changed][started];
        }

        int limit = tight ? digits[pos] - '0' : 9;
        int answer = 0;

        for (int d = 0; d <= limit; d++) {
            bool nextTight = tight && (d == limit);

            if (!started && d == 0) {
```

```

        answer += dfs(pos + 1, changed, false, nextTight);
    } else {
        if (!isValidDigit(d)) {
            continue;
        }

        bool nextChanged = changed || isChangingDigit(d);

        answer += dfs(pos + 1, nextChanged, true,
nextTight);
    }
}

if (!tight) {
    seen[pos][changed][started] = true;
    memo[pos][changed][started] = answer;
}

return answer;
}

int rotatedDigits(int n) {
    digits = to_string(n);
    memset(seen, false, sizeof(seen));

    return dfs(0, false, false, true);
}
};

```

28. Common Digit DP Edge Cases

28.1 $n = 0$

Decide whether 0 should count.

Some problems count non-negative integers.

Some count positive integers only.

28.2 Leading Zeros

Ask:

Should leading zeros affect digit sum, digit count, mask, parity, or previous digit?

Usually no.

28.3 Empty Number

If `started == false` at the end, this represents number 0 or an empty number.

Return carefully.

28.4 Low = 0

For range:

```
solve(high) - solve(low - 1)
```

If low is 0, low - 1 is invalid.

Handle separately.

28.5 String Bounds

When bounds are strings, use string subtract-one carefully.

28.6 Modulo

Counting problems often require modulo 1,000,000,007.

28.7 Tight Memoization

Do not blindly memoize tight states unless your memo key includes enough information.

Usually memoize only when:

```
tight == false
```

29. Common Digit DP Mistakes

Mistake 1 — Wrong Tight Update

Correct:

```
bool nextTight = tight && (d == limit);
```

Mistake 2 — Counting Leading Zeros as Digits

For no repeated digits, do not mark digit 0 as used before number starts.

Mistake 3 — Forgetting started in Base Case

If counting positive numbers, base case should often be:

```
return started && valid;
```

Mistake 4 — Memoizing Without considering started

If started affects future behavior, it must be part of memo state.

Mistake 5 — Incorrect Range Formula

Correct:

```
count [low, high] = solve(high) - solve(low - 1)
```

Mistake 6 — Not Resetting Memo Between solve Calls

For range problems, solve(high) and solve(low - 1) use different digit strings.

Reset memo each time.

30. Interview Explanation Templates

30.1 General Digit DP

I build numbers digit by digit from left to right. The state tracks the current position, whether the prefix is still tight to the upper bound, and any extra information required by the problem.

30.2 Tight Explanation

If the prefix so far equals the prefix of n, the current digit cannot exceed the corresponding digit in n. Otherwise, I can choose any digit from 0 to 9.

30.3 Leading Zero Explanation

I use a started flag to distinguish leading zeros from actual digits. This prevents leading zeros from incorrectly affecting digit masks, previous digit checks, or digit counts.

30.4 Range Explanation

I define solve(x) as the number of valid integers from 0 to x. Then the answer for [low, high] is solve(high) minus solve(low - 1).

30.5 Used Mask Explanation

For no repeated digits, I use a 10-bit mask where bit d indicates whether digit d has already appeared in the number.

31. Complexity Patterns

Problem Type	Time	Space
Basic digit sum	$O(d * \text{sum} * 10)$	$O(d * \text{sum})$
Used digit mask	$O(d * 1024 * 10)$	$O(d * 1024)$
Modulo k	$O(d * k * 10)$	$O(d * k)$
Previous digit	$O(d * 11 * 10)$	$O(d * 11)$
Digit sum range	$O(d * \text{maxSum} * 10)$	$O(d * \text{maxSum})$
Beautiful integer	$O(d * d * d * k * 10)$	depends on state
String bound length m	$O(m * \text{state} * 10)$	$O(m * \text{state})$

Here d means number of digits.

32. Expanded Practice Set

32.1 Must-Do Medium

- 16. Number of Digit One
- 17. Count Special Integers
- 18. Numbers With Repeated Digits
- 19. Numbers At Most N Given Digit Set
- 20. Rotated Digits
- 21. Non-negative Integers Without Consecutive Ones
- 22. Count Numbers With Unique Digits

32.2 Strong Interview Practice

- 23. Count of Integers
- 24. Count Stepping Numbers in Range
- 25. Number of Beautiful Integers in Range
- 26. Count numbers with exact digit sum
- 27. Count numbers divisible by k
- 28. Count numbers with no adjacent equal digits
- 29. Count binary strings without consecutive ones

32.3 Hard / Stretch

- 30. Count Special Integers with additional constraints
 - 31. Count numbers with digit sum and modulo constraints
 - 32. Count numbers with equal odd/even digits and divisibility
 - 33. Count stepping numbers in very large string range
 - 34. Count numbers with digit frequency constraints
 - 35. Digit DP with automaton / forbidden substring
-

33. Suggested Study Order

Step 1 — Basic Digit DP

Start with:

- 36. Count numbers with exact digit sum
- 37. Number of Digit One
- 38. Rotated Digits

Goal:

Understand pos, tight, and base cases.

Step 2 — Leading Zeros and Started

Solve:

- 39. Count positive numbers with digit sum
- 40. Numbers At Most N Given Digit Set
- 41. No adjacent equal digits

Goal:

Understand started.

Step 3 — Digit Mask

Solve:

- 42. Count Special Integers
- 43. Numbers With Repeated Digits

Goal:

Understand digit masks.

Step 4 — Modulo State

Solve:

- 44. Count numbers divisible by k
- 45. Beautiful Integers

Goal:

Understand remainder transitions.

Step 5 — Hard Range String Problems

Solve:

- 46. Count of Integers
- 47. Count Stepping Numbers in Range

Goal:

Understand string bounds and subtract-one helpers.

34. Pattern Decision Table

Problem Clue	State
digit sum equals target	pos, sum, tight
digit sum in range	pos, sum, tight
no repeated digits	pos, mask, started, tight
repeated digits	total - noRepeated
divisible by k	pos, remainder, started, tight
adjacent digit condition	pos, previousDigit, started, tight
binary no consecutive ones	pos, previousBit, tight
stepping number	pos, previousDigit, started, tight
allowed digit set	pos, started, tight
equal odd/even digits	pos, odd, even, remainder, started, tight
count in [low, high]	solve(high) - solve(low - 1)

35. Mini Problem 1 — Count Numbers With Digit Sum At Most S

Instead of exact sum, base case checks:

```
return started && sum <= S;
```

State remains:

```
pos, sum, started, tight
```

If sum already exceeds S, return 0.

36. Mini Problem 2 — Count Numbers Containing Digit 7

State:

```
pos, hasSeven, started, tight
```

At each digit:

```
nextHasSeven = hasSeven || (d == 7);
```

Base:

```
return started && hasSeven;
```

37. Mini Problem 3 — Count Numbers Without Digit 4

At each digit, skip 4:

```
if (d == 4) {  
    continue;  
}
```

State may only need:

pos, started, tight

38. Mini Problem 4 — Count Numbers With Product of Digits Divisible by k

State:

pos, productMod, started, tight

Transition:

```
nextProductMod = (productMod * d) % k;
```

But be careful with digit 0:

If started and d is 0, product becomes 0.

This type of problem often needs careful handling.

39. Debugging Digit DP

If your Digit DP fails, check:

- 48. What does dfs state mean exactly?
- 49. Is tight updated correctly?
- 50. Are leading zeros handled correctly?
- 51. Should 0 be counted?
- 52. Is started part of memoization?
- 53. Is memo reset between solve calls?
- 54. Are you memoizing only when tight is false?
- 55. Is low - 1 handled correctly?
- 56. Are digit sums pruned safely?
- 57. Are modulo states updated correctly?
- 58. Are repeated digits tracked after number starts only?

40. What to Master Before Chapter 43

Before moving to DP Optimization Patterns, make sure you can:

- Explain tight
 - Explain started
 - Write solve(n)
 - Convert range [low, high] into solve(high) - solve(low - 1)
 - Track digit sum
 - Track previous digit
 - Track used digit mask
 - Track remainder modulo k
 - Handle string bounds
 - Subtract one from a numeric string
 - Memoize only reusable states
 - Solve Count Special Integers
 - Solve Count of Integers
 - Solve Stepping Numbers
 - Solve Non-negative Integers Without Consecutive Ones
-

41. Final Cheat Sheet

Basic Digit DP State

```
dfs(pos, tight, started, state)
```

Limit

```
int limit = tight ? digits[pos] - '0' : 9;
```

Tight Update

```
bool nextTight = tight && (d == limit);
```

Started Update

```
bool nextStarted = started || d != 0;
```

Digit Sum Update

```
nextSum = sum + d;
```

Remainder Update

```
nextRem = (rem * 10 + d) % k;
```

Used Digit Mask

```
if (mask & (1 << d)) {
    continue;
}

nextMask = mask | (1 << d);
```

Range Count

```
answer = solve(high) - solve(low - 1);
```

Memoization Rule

Usually memoize only when:

```
tight == false
```

42. Core Takeaway

Digit DP is DP over digits.

Use it when the range is huge but the number of digits is small.

The core state is usually:

```
position  
tight  
started  
problem-specific state
```

The hardest parts are:

- understanding tight
- handling leading zeros
- choosing the right state
- writing the base case correctly
- solving ranges with `solve(high) - solve(low - 1)`

Once you master these, many scary-looking counting problems become systematic.

Digit DP is not about memorizing solutions.

It is about asking:

```
What information do I need while building the number digit by digit?
```

That information becomes your DP state.